

Experiment:13

Construct a C program to implement various memory allocation strategies.

Code:-

```
#include <stdio.h>

void bestfit(int mp[], int p[], int m, int n)
{
    int j = 0;
    for (int i = 0; i < n; i++)
    {
        if (mp[i] >= p[j])
        {
            printf("\nProcess %d fits in Memory Partition %d",
                p[j], mp[i]);

            mp[i] = mp[i] - p[j];
            j++;
            i--;
        }
        if (j == m)
            break;
    }
    for (int i = j; i < m; i++)
    {
        printf("\nProcess %d must wait", p[i]);
    }
}
```

```
void sort(int a[], int n)  
{  
    int t;  
    for (int i = 0; i < n - 1; i++)  
    {  
        for (int j = i + 1; j < n; j++)  
        {  
            if (a[i] > a[j])  
            {  
                t = a[i];  
                a[i] = a[j];  
                a[j] = t;  
            }  
        }  
    }  
}
```

```
void rsort(int a[], int n)  
{  
    int t;  
    for (int i = 0; i < n - 1; i++)  
    {  
        for (int j = i + 1; j < n; j++)  
        {  
            if (a[i] < a[j])  
            {  
                t = a[i];
```

```
        a[i] = a[j];  
        a[j] = t;  
    }  
}  
}  
}
```

```
void firstfit(int mp[], int p[], int m, int n)
```

```
{  
    bestfit(mp, p, m, n);  
}
```

```
void bestFitMethod(int mp[], int p[], int m, int n)
```

```
{  
    sort(mp, n);  
    sort(p, m);  
    bestfit(mp, p, m, n);  
}
```

```
void worstfit(int mp[], int p[], int m, int n)
```

```
{  
    rsort(mp, n);  
    sort(p, m);  
    bestfit(mp, p, m, n);  
}
```

```
int main()
```

```
{  
    int m, n, mp[20], p[20], ch;
```

```
printf("Number of Memory Partitions: ");
scanf("%d", &n);
printf("Number of Processes: ");
scanf("%d", &m)
printf("Enter Memory Partition Sizes:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &mp[i]);

printf("Enter Process Sizes:\n");
for (int i = 0; i < m; i++)
    scanf("%d", &p[i]);
printf("\n1. First Fit\n2. Best Fit\n3. Worst Fit\n");
printf("Enter your choice: ");
scanf("%d", &ch);
switch (ch)
{
    case 1:
        firstfit(mp, p, m, n);
        break;
    case 2:
        bestFitMethod(mp, p, m, n);
        break;
    case 3:
        worstfit(mp, p, m, n);
        break;

    default:
```

```
        printf("Invalid Choice");  
    }  
  
    return 0;  
}
```

Output:

```
Number of Memory Partitions: 3  
Number of Processes: 2  
Enter Memory Partition Sizes:  
4  
5  
6  
Enter Process Sizes:  
8  
9  
  
1. First Fit  
2. Best Fit  
3. Worst Fit  
Enter your choice: 7  
Invalid Choice  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```